



CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
RELATÓRIO TÉCNICO DE ATIVIDADE

LUCAS EDUARDO TRAVASSOS MACHADO

JOINVILLE/SC
2026

1. O Event Loop no Node.js e o Atendimento Multiconexão em Single-Thread

O Node.js executa o código JavaScript em uma única thread principal (*single-thread*). Diferentemente de plataformas tradicionais como o Java ou o C#, que tipicamente alocam uma nova thread do sistema operacional para cada requisição HTTP recebida, o Node.js adota uma abordagem de arquitetura orientada a eventos e I/O não bloqueante (NODE.JS DESIGN PATTERNS, 2020). A criação e a manutenção de múltiplas threads consomem recursos significativos de memória RAM e causam custos de processamento com trocas de contexto (*context switching*). O Node.js elimina essa sobrecarga ao processar requisições através de um mecanismo contínuo chamado *Event Loop*.

O *Event Loop* monitora constantemente a pilha de execução (*call stack*) e a fila de eventos. Quando o Node.js identifica uma operação de Entrada/Saída (E/S) potencialmente demorada, tal como consultas a bancos de dados, leitura/escrita de arquivos no disco ou requisições de rede, ele não paralisa a thread principal. A operação é delegada ao sistema operacional ou à fila interna de threads mantida pela biblioteca C/C++ *libuv* (MOZILLA DEVELOPER NETWORK, 2025). O código registra uma função de retorno (*callback*) e a thread principal é liberada imediatamente para atender outras requisições recebidas (NODE.JS ORG, 2026a). Quando a operação assíncrona é concluída em segundo plano, a *libuv* coloca o *callback* correspondente na fila do *Event Loop*, que o resgata e executa assim que a pilha principal estiver desocupada.

Exemplo de Aplicação Prática: Esse comportamento é visível ao executar o comando `docker run hello-world` no Docker Desktop. A interface gráfica e o terminal de linha de comando não travam durante o download (*pull*) da imagem na rede. O download acontece em segundo plano de forma assíncrona, e o resultado só é exibido na tela no instante exato em que a resposta da requisição é concluída.

2. Processamento Síncrono x Assíncrono e os Riscos do Bloqueio da Thread

Em uma operação síncrona, o fluxo de execução do programa fica estritamente parado aguardando o término do processamento para avançar para as linhas seguintes. Em uma operação assíncrona, a instrução é disparada e o código prossegue imediatamente, definindo o tratamento da resposta futura via *callbacks*, *Promises* ou construções *async/await* (FLANAGAN, 2020).

Devido à arquitetura *single-thread* do Node.js, a execução de uma rotina síncrona pesada, como cálculos matemáticos complexos em loops ou leitura síncrona de arquivos grandes via `fs.readFileSync`, paralisa toda a aplicação. Enquanto a thread principal estiver ocupada com essa tarefa bloqueante, nenhuma outra requisição poderá ser recebida ou respondida pelo servidor (NODE.JS ORG, 2026b). Em ambientes de produção de APIs web, uma única instrução bloqueante de um usuário paralisa o atendimento de todos os demais usuários simultâneos, configurando um risco grave de disponibilidade.

Exemplo de Aplicação Prática: Ao executar localmente `npx tsx src/index.ts` com um simples `console.log`, o retorno é instantâneo. Em um cenário de API conectada a um banco de dados relacional (como PostgreSQL ou MySQL), efetuar essa busca de maneira síncrona e bloqueante deixaria o servidor totalmente inoperante e incapaz de responder até mesmo a chamadas de monitoramento (*health check*) durante todo o tempo de resposta do banco.

3. TypeScript e a Resolução da Tipagem Dinâmica

O TypeScript é um superset estritamente tipado do JavaScript, desenvolvido e mantido pela Microsoft (MICROSOFT, 2026). Como superset, todo código JavaScript sintaticamente válido também é um código TypeScript válido, adicionando uma camada de checagem estática de tipos sobre a linguagem base.

No JavaScript puro, a tipagem é dinâmica, o que significa que variáveis podem mudar de tipo livremente e inconsistências só são descobertas em tempo de execução (*runtime*), frequentemente em ambiente de produção após o sistema ser entregue aos usuários. O TypeScript resolve esse problema ao introduzir a checagem em tempo de compilação (*build time*). O compilador do TypeScript (tsc) analisa o código durante o desenvolvimento e avisa imediatamente se houver incompatibilidades de tipos, chamadas de métodos inexistentes ou retornos inconsistentes (BIERMAN et al., 2014).

Exemplo de Aplicação Prática: A inicialização de um projeto via `npx tsc --init` gera o arquivo `tsconfig.json`. Ao habilitar a opção `"strict": true` nesse arquivo de configuração, o compilador passa a exigir tipagens explícitas e validações rigorosas de valores nulos e indefinidos, prevenindo falhas clássicas antes que o código chegue ao ambiente de testes ou produção.

4. Sistemas de Módulos: CommonJS versus ES Modules (ESM)

O Node.js possui dois ecossistemas distintos para organização e importação de módulos: o **CommonJS** e o **ES Modules (ESM)**.

O CommonJS é o padrão histórico do Node.js, utilizando `require()` para importar bibliotecas e `module.exports` para exportar funcionalidades. Ele carrega os módulos de forma síncrona diretamente do disco (NODE.JS ORG, 2026c). Por outro lado, o ES Modules (ESM) é o padrão oficial introduzido pelo ECMAScript (ES6), utilizando a sintaxe `import` e `export`. O ESM permite carregamento assíncrono e foi projetado para unificar a modularização no navegador e no servidor (MDN, 2025).

Os problemas de compatibilidade ocorrem porque grande parte da base legada de pacotes do ecossistema NPM foi escrita originalmente em CommonJS. Quando um projeto configurado em ESM tenta importar uma biblioteca em CommonJS sem os devidos tratamentos de interoperabilidade, ocorrem falhas na resolução de escopo. O Node.js identifica o formato utilizado examinando a extensão dos arquivos (`.cjs` vs `.mjs`) ou pelo campo `"type"` definido no manifesto do projeto.

Exemplo de Aplicação Prática: No arquivo `package.json` gerado pelo comando `npm init -y`, a propriedade `"type": "commonjs"` é definida por padrão. Isso significa que o runtime interpretará os arquivos JavaScript locais como CommonJS, exigindo configurações específicas caso se deseje migrar para ESM nativo (`"type": "module"`).

5. Comparativo Entre Frameworks Backend: Express, Fastify e NestJS

A escolha do framework backend impacta a arquitetura, a velocidade de desenvolvimento e a performance do projeto. A tabela abaixo resume as principais características técnicas das três opções mais populares do mercado:

Critério	Express	Fastify	NestJS
Filosofia / Arquitetura	Minimalista, flexível e não-opinativo (EXPRESS, 2026) .	Minimalista, focado em alta performance e schemas (FASTIFY, 2026) .	Altamente opinativo, modular com injeção de dependência (NESTJS, 2026) .
Desempenho e Validação	Desempenho moderado; validações dependem de libs externas.	Alta vazão de dados; validação nativa com JSON Schema.	Baseado em Express ou Fastify; utiliza TypeScript nativo.
Cenário Recomendado	Protótipos, projetos de aprendizado e APIs de menor porte.	APIs de altíssimo tráfego e microsserviços de baixa latência.	Sistemas corporativos complexos e equipes grandes.

O **Express** oferece liberdade total para a equipe organizar o código, sendo ideal para entender o funcionamento cru de rotas e middlewares. O **Fastify** é ideal para cenários onde cada milissegundo de resposta é crítico, fornecendo estrutura de validação integrada. Já o **NestJS** impõe uma estrutura arquitetural padronizada (semelhante ao Angular e ao Spring Boot do Java), facilitando a manutenção e a entrada de novos desenvolvedores em projetos corporativos.

6. O Papel do NVM no Gerenciamento de Versões do Node.js

O *Node Version Manager* (NVM) é uma ferramenta utilitária que permite instalar, gerenciar e alternar entre múltiplas versões do Node.js na mesma máquina sem a necessidade de desinstalar o ambiente global (COREY BUTLER, 2026; NVM SH, 2026) .

Nas empresas, é comum manter projetos legados em versões anteriores do Node.js enquanto novas aplicações são desenvolvidas utilizando as versões mais recentes. O NVM é considerado uma prática essencial porque evita o conflito de dependências entre projetos e elimina a necessidade de reinstalar a plataforma a cada troca de contexto de trabalho.

Exemplo de Aplicação Prática: Utilizando o nvm-windows, a execução dos comandos nvm install lts e nvm use lts ativa a versão LTS mais recente (como a 24.19.0). A utilização de alianças de versão (como --lts) previne que a equipe fique presa a versões desatualizadas ou sem suporte de segurança de longo prazo.

7. Containers Docker versus Máquinas Virtuais (VMs)

A diferença central entre um container Docker e uma Máquina Virtual (VM) reside no nível de virtualização:

- **Máquina Virtual (VM):** Virtualiza o hardware completo do computador sobre um *Hypervisor*. Cada VM contém uma cópia completa de um sistema operacional, incluindo kernel, drivers e utilitários. Isso torna as VMs mais pesadas em termos de uso de memória, disco e tempo de boot (DOCKER, 2026) .
- **Container Docker:** Virtualiza apenas o ambiente de execução da aplicação no nível do sistema operacional. O container compartilha o kernel do SO hospedeiro, isolando apenas os processos, bibliotecas e dependências (DOCKER, 2026) .

Exemplo de Aplicação Prática: Ao criar um Dockerfile utilizando a imagem leve

node:24-alpine em um ambiente Windows 11 com Docker Desktop e backend Hyper-V, o container roda de forma quase instantânea ao executar `docker run --rm hello-node`. Isso demonstra a leveza do container frente aos minutos exigidos para inicializar uma VM completa.

Referências Bibliográficas

BIERMAN, Gavin; ABADI, Martín; TSETSEKAS, Panagiotis. Understanding TypeScript. In: **European Conference on Object-Oriented Programming (ECOOP)**. Springer, Berlin, Heidelberg, 2014. p. 257-281.

COREY BUTLER. **nvm-windows: A node.js version manager for Windows**. Disponível em: <<https://github.com/coreybutler/nvm-windows>>. Acesso em: 28 ago. 2026.

DOCKER. **What is a Container?** Docker Docs. Disponível em: <<https://docs.docker.com/get-started/overview/>>. Acesso em: 28 ago. 2026.

EXPRESS. **Express - Fast, unopinionated, minimalist web framework for Node.js**. Disponível em: <<https://expressjs.com/>>. Acesso em: 28 ago. 2026.

FASTIFY. **Fastify - Fast and low overhead web framework, for Node.js**. Disponível em: <<https://fastify.dev/>>. Acesso em: 28 ago. 2026.

FLANAGAN, David. **JavaScript: O Guia Definitivo**. 7. ed. Porto Alegre: Bookman, 2020.

MICROSOFT. **TypeScript Documentation**. Disponível em: <<https://www.typescriptlang.org/docs/>>. Acesso em: 28 ago. 2026.

MOZILLA DEVELOPER NETWORK (MDN). **The event loop**. MDN Web Docs. Disponível em: <<https://developer.mozilla.org/en-US/docs/Web/JavaScript/EventLoop>>. Acesso em: 28 ago. 2026.

NESTJS. **NestJS - A progressive Node.js framework**. Disponível em: <<https://docs.nestjs.com/>>. Acesso em: 28 ago. 2026.

NODE.JS DESIGN PATTERNS. CASCIARO, Mario; MAMOULIANOS, Luciano. **Node.js Design Patterns**. 3. ed. Packt Publishing, 2020.

NODE.JS ORG. **The Node.js Event Loop, Timers, and process.nextTick()**. Disponível em: <<https://nodejs.org/en/docs/guides/event-loop-timers-and-nexttick/>>. Acesso em: 28 ago. 2026.

NODE.JS ORG. **Don't Block the Event Loop (or Worker Pool)**. Disponível em: <<https://nodejs.org/en/docs/guides/dont-block-the-event-loop/>>. Acesso em: 28 ago. 2026.

NODE.JS ORG. **Modules: CommonJS modules**. Disponível em: <<https://nodejs.org/api/modules.html>>. Acesso em: 28 ago. 2026.

NVM SH. **nvm: Node Version Manager**. Disponível em: <<https://github.com/nvm-sh/nvm>>. Acesso em: 28 ago. 2026.